
Ribbon Catches Bloom: Pleated Construction at Bloom Speed

Brian Sam-Bodden
Integrallis Software

Abstract

Probabilistic membership filters (Bloom filters and their modern successors) answer a single question: is this key possibly in this set? Storage systems keep one in front of nearly every file on disk, so a lookup that would find nothing can skip the read entirely. Ribbon and binary-fuse filters answer that question as accurately as a Bloom filter in far less memory, within a few percent of the information-theoretic minimum. Bloom filters still win in practice anyway, and the reason is build cost. Filters are immutable; compaction rebuilds them constantly; and a ribbon filter takes roughly four times the CPU of a Bloom filter to construct.

This paper closes most of that gap for homogeneous ribbon filters. A single counting pass groups keys by the region of the filter they touch — a weaker, roughly four-times-cheaper stand-in for the full sort prior work relies on — and that alone makes sequential construction $2.05\text{--}2.24\times$ faster at 100–400 million keys. One property keeps it safe, and we prove it before leaning on it and then re-check it on every run: the finished filter is bit-for-bit identical no matter what order keys arrive in. Any reordering, a parallel build included, can therefore vouch for itself with a single checksum against the sequential result. Behind the speedup is one rule about modern processors, and two failures are what pinned it down. Batching and layout tricks do nothing for filter queries, or for Bloom construction, because those memory stalls are independent and out-of-order hardware already overlaps them. Ribbon construction is the exception. Its stalls form a dependent chain, each elimination step waiting on the one before, and a dependent chain is the one memory cost hardware cannot hide but reordering can. With 16 threads a ribbon filter then builds about as fast as a single thread fills a Bloom filter. On the hardware we tested that is the quickest construction we know of, and it takes away the last real reason to prefer Bloom: the filter that needs a quarter less memory at the same accuracy no longer costs more to build. Dropped into RocksDB’s shipped standard-ribbon builder, the same one-pass partition makes construction $1.8\times$ faster with a bit-for-bit identical filter, so the mechanism carries over to a production engine untouched.

1 Introduction

A membership filter is a small in-memory summary of a large key set. Ask it whether a key is present and it answers “definitely not” or “probably yes.” The first answer is the one worth having: a database that keeps a filter in front of each data file skips reading any file whose filter says no. Bloom filters [1] have done this for five decades, at a cost of about 44% more memory than the theoretical minimum for a given error rate. A newer generation — xor, binary-fuse, and ribbon — closes that gap to within 10%, and to under 1% in ribbon’s bumped variants [2, 3].

There is a catch, and it is this paper’s subject. The new filters are static: built once over a fixed key set, never updated. That suits write-optimized databases well enough, since their data files are immutable

and background compaction rewrites them continuously. But every rewrite also rebuilds the file’s filter, so build speed matters as much as query speed — and building is exactly where the new filters lose. A ribbon filter costs about four times the CPU of a Bloom filter (RocksDB measures 140 vs. 32 ns per key [4]), so production systems keep Bloom as the default and save the space-efficient kind for data they rarely rewrite. The people closest to these structures say so themselves. The parallel-BuRR authors report that “a large part of the construction time is spent on parallel sorting” [5], and the most recent proposals concede they build several-fold slower than the best existing builders [6].

Which parts of that cost are real — fixed by the hardware — and which can be won back? Three experiments answer, and they collapse to one rule: optimization pays exactly where memory stalls form data-dependent chains. Independent stalls are already overlapped by out-of-order execution and software prefetch; dependent chains defeat both.

Contributions.

1. Ribbon construction gets dramatically cheaper (§4). Homogeneous ribbon banding at scale burns 5.8 DRAM misses per key in dependent row-reduction chains, and those chains are exactly what prefetch cannot hide. A single counting pass that groups keys into L2-sized start-windows (approximate order, nothing more) recovers 98.3–98.4% of the miss reduction that the full start-position sort (SGAUSS [7], BuRR [3]) buys, at 5.5–6.1 ns/key where BuRR’s own `ips2ra` sorter costs 23.9–28.1. End to end that makes construction 2.05–2.24× faster than the reference prefetch-pipelined build. Window size barely matters; totals stay within 13% across 2^{13} through 2^{20} slots.
2. Banding turns out to be order-independent, and that observation carries real weight (§6). With first-coefficient-always-one pivoting, the solved filter is bit-identical under any insertion order. We prove this for linearly independent rows and saw it hold in 54 of 54 runs. So reordering is output-neutral by construction, and a slot-range parallel variant that defers boundary keys (at most 0.23% of them) can verify itself with a checksum: 4.25× faster banding at 8 threads, 7.56× at 16, against its own single-thread run (3.5× and 6.3× against the best sequential banding). The total lands at 11.9 ns/key at 100M keys.
3. The rule above came from failing twice, and the failures are load-bearing (§3). We could not make batched probing win, and the reason turned out to matter more than the attempt. Miss-dominated point probes run into the per-core limit on outstanding misses; software prefetch was already at 88% of that ceiling when we measured it. Cross-key SIMD batching loses to plain per-key probing even with every address L1-resident. Bulk Bloom construction fell the same way, for the same underlying reason: its misses are independent, so the memory system hides them. Radix-partitioned construction in the style of Schmidt et al. [8] topped out at 1.45–1.71× over per-key insertion, a prefetch loop that RocksDB ships (but never evaluated) recovered most of that with zero transient memory, and our read-free register-aggregated apply (the idea we had the most hope for) came in 1.5× slower than partitioned RMW, because partitioning had already turned the target misses into L2 hits.
4. Along the way we ended up producing the first actual evaluation of the construction prefetch RocksDB ships behind a “TODO: verify/validate” comment (§7). It is worth 1.5–1.7× on unsorted banding. Partitioning mostly subsumes it.
5. The technique transfers to production unchanged (§8). Patched into RocksDB’s shipped standard-ribbon ($w=128$) builder, one counting pass makes construction 1.8× faster at scale with a bit-for-bit identical filter (25× fewer banding cache-misses at the same instruction count) and removes a scale-growing slice of compaction CPU at zero cost to the read or write path.

All measurements come from an i9-14900HX (AVX2, no AVX-512). Comparative numbers are produced by the field’s reference benchmark suites, run unmodified at pinned versions: `fastfilter_cpp` (its ribbon implementations are the reference code of ribbon’s creator, Peter Dillinger) and BuRR. The only benchmark code we wrote is a thin driver that permutes the input order around the unmodified kernel, so every measured difference is attributable to input order alone. The repository holds the raw output of every run and a script that re-derives each number in this paper from it. We replicate the ARM results on two additional microarchitectures (§5).

2 Background

Ribbon banding. A ribbon filter solves a banded GF(2) system. Each key hashes to a start column s and a w -bit coefficient row, and insertion XOR-reduces that row against whatever is already stored until an empty slot absorbs it. The lineage matters here. Dietzfelbinger and Walzer showed that sorting rows by start position makes banded elimination linear (SGAUSS). The ribbon paper’s whole pitch was that you don’t need the sort: insert on the fly, in arbitrary order. BuRR then brought the sort back (ips2ra, by bucket) because bumping needs it. Homogeneous ribbon, the variant we target, never fails construction and stores no per-key result rows.

What the filter stores and how a query works. A ribbon filter is an $m \times r$ bit matrix Z . Hashing maps each key k to three values: a start position $s(k)$, a w -bit coefficient row $c(k)$ whose first bit is always 1, and r result bits $b(k)$. The filter represents k by satisfying one linear equation over GF(2): $c(k) \cdot Z = b(k)$, with $c(k)$ occupying columns $[s(k), s(k)+w)$. A query recomputes s , c , and b and answers “present” exactly when $c \cdot Z = b$. A key that was never inserted passes that test by chance with probability 2^{-r} , which is the false-positive rate; at $r = 7$ that is just under 0.8%.

Why banding is local. Construction solves the linear system incrementally: each insertion XOR-reduces its row against stored rows until an empty slot absorbs it. Because a key’s coefficients span only w consecutive columns, all of the work for that key happens in a short run of slots beginning at $s(k)$, and the expected run is short [2]. Every technique in this paper rests on that fact: a key belongs to the neighborhood of its start position, so grouping keys by start region confines banding’s memory traffic to one region at a time, and a thread that owns a range of slots can process every key whose run provably stays inside it.

The homogeneous variant. Homogeneous ribbon stores no per-key result rows during banding and never fails: a row that reduces to zero is dropped, trading a small false-positive elevation for guaranteed completion. Our configuration throughout is $w = 64$, $r = 7$, and $m \approx 1.09n$ slots: 7.63 bits per key at a measured 0.81% false-positive rate.

Binary fuse construction partially sorts keys by segment explicitly to reduce cache misses [9]; our Phase-0 characterization measures that fix directly (plain xor: 9.45 misses/key; binary fuse: 1.08).

3 Where layout tricks cannot work

This section is about two optimizations that failed. Both got their best case: the batched probe was timed with every address already in cache, and the read-free builder was the design its own experiment was built around, each protocol fixed before any run. Both lost anyway. That is what makes them worth reporting: an optimization that cannot win even when the conditions are chosen to favor it is telling you something about the hardware, not the setup.

3.1 Point probes are MLP-bound

Experiment E0 compared probe strategies over identical split-block Bloom filters (Parquet geometry), 32KiB up to 512MiB, with the protocol frozen before any timing run. Our vertical, lane-structured batch probe does auto-vectorize from plain Rust. The packed-SIMD disassembly is committed. It still loses. At every size, even with every address forced L1-hot, per-key scalar probing wins (665 vs. 555 Mops/s), because a split-block check is already SIMD within a single key, and an AVX2 gather cannot beat eight independent scalar loads that the out-of-order window overlaps for free. The RAM-scale picture is starker: the identical instruction stream runs 5–6 $\times$ faster when L1-hot. Probing is bound by memory-level parallelism. Prefetch was already at roughly 88% of the single-core outstanding-miss ceiling. We measured it. There is no compute bottleneck left for a layout to remove.

3.2 Blocked-Bloom construction is prefetch-hidable

Experiment E1a measured five bulk builders at 1M to 429M keys, gated on bit-identical output (OR is commutative, so any deviation is a bug). Per-key insertion does degrade at scale (one random read-modify-write miss per key, 11.5 ns/key at 100M, IPC 0.48), so there was a real cost to attack. The attacks mostly failed. Radix-partitioned construction after Schmidt et al. [8] reached only 1.45–1.71 $\times$. A software-pipelined prefetch insert, the shape RocksDB ships, hit 6.1–8.9 ns/key with zero transient

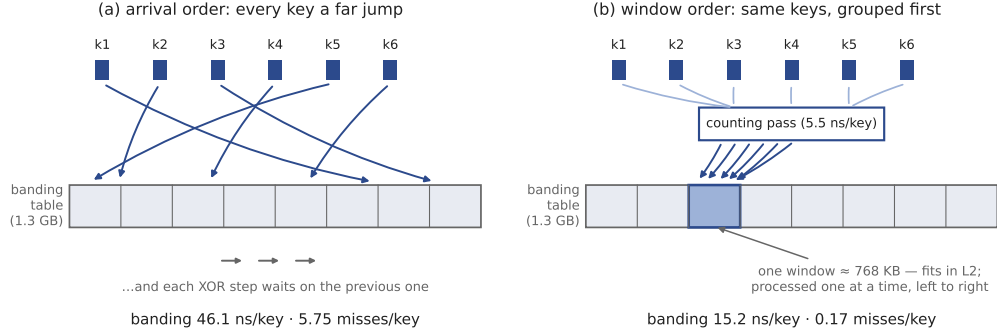


Figure 1: Why insertion order is the whole game. (a) In arrival order, every key starts its elimination at a random position in a table three orders of magnitude larger than cache, and the steps within each insertion form a dependent chain no prefetcher can run ahead of. (b) A single counting pass groups the same keys by table window; banding then works inside one cache-resident window at a time. Binary fuse filters apply the same remedy to their counter scatter via a segment sort (§2). Numbers: 100M keys, means of 3 runs.

memory and actually beat partitioning up to 100M keys. And the variant we designed this experiment around — a read-free apply that partitions pre-merged (block, mask) pairs, OR-aggregates each block in registers, and issues exactly one store per block without ever reading filter memory — came in $1.5\times$ slower than partitioned RMW. Non-temporal stores changed nothing. In hindsight the accounting is obvious: partitioning had already turned the expensive misses into L2 hits, so the extra grouping pass was spending real time to save cheap loads. Bloom construction misses are independent, and the memory system hides independent misses better than we could.

4 Partition-instead-of-sort for ribbon

4.1 The target, measured

We first measured where ribbon construction spends its time, using the reference benchmark suite unmodified (three repetitions). At 100 million keys, homogeneous ribbon construction costs 50.6 ns per key, and the hardware counters say why: the processor completes only 0.64 instructions per cycle and takes 5.81 cache misses per key, which is to say it spends most of its time waiting on memory. The reason is structural. Each key starts its elimination at an effectively random position in a 1.3GB table, and every reduction step needs the result of the previous XOR before it can even compute which slot it touches next. A prefetcher cannot run ahead of a chain like that. This is exactly the opposite of Bloom construction, where each key’s memory accesses are independent and the hardware overlaps them for free.

4.2 Design

We only permute the input. One counting pass buckets keys by $\text{floor}(s / 2^{16})$ (windows of 2^{16} slots, roughly 768KiB of banding state, under half an L2), recomputing the hash on the fly instead of materializing pairs; a second pass writes the keys back out in window order. The untouched reference kernel, prefetch pipeline and all, then consumes the permuted array. Everything downstream (banding, back-substitution, queries) is that reference code, not ours; the speedup is therefore attributable to reordering alone. Figure 1 shows the access-pattern change this buys.

4.3 Results

Table 1 (100M keys; 400M in artifacts). The partition pass costs 5.5-6.1 ns/key against 23.9-28.1 for ips2ra in our driver (17.3 inside BuRR’s own bench); banding lands within 13% of banding-after-full-sort (15.2 vs. 13.4 ns/key), i.e., the partition recovers 98.3-98.4% of the sort miss reduction ($5.75 \rightarrow 0.17$ vs. 0.075 misses/key). End-to-end we get 24.2 vs. 49.6 (reference) and 40.7 (best full sort):

Table 1: Homogeneous ribbon construction, 100M keys, ns/key (mean $\pm$ sample stdev over 3 reps; parallel rows 2 reps; raw artifacts in repository). Banding perf counters scoped to the banding phase.

strategy	reorder	banding	total	miss/key
unsorted, no prefetch	—	76.9 ± 4.3	80.8 ± 4.2	5.81
unsorted + prefetch (ref.)	—	46.1 ± 1.4	49.6 ± 1.4	5.75
full sort (std::sort)	76.8 ± 7.1	15.5 ± 2.2	96.1 ± 8.9	0.074
full sort (LSD radix)	23.4 ± 1.9	14.7 ± 0.8	41.7 ± 1.2	0.076
full sort (ips2ra)	23.9 ± 0.1	13.4 ± 0.1	40.7 ± 0.3	0.075
partitioned (ours)	5.5 ± 0.1	15.2 ± 0.1	24.2 ± 0.2	0.170
parallel, 8T (ours)	5.5	4.3 ± 0.4	14.6 ± 2.1	—
parallel, 16T (ours)	5.7	2.4 ± 0.4	11.9 ± 0.8	—

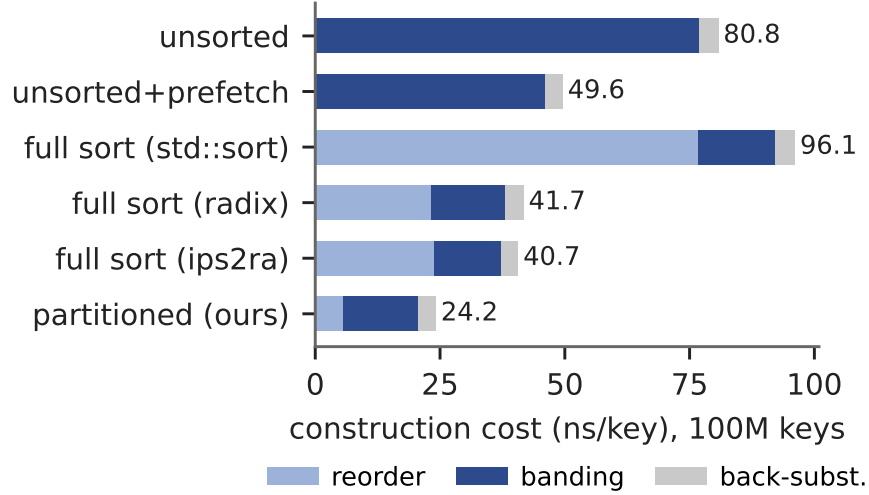


Figure 2: Construction cost by phase, 100M keys. The partition pass replaces the sort at $4\times$ lower reorder cost while banding stays within 13% of banding-after-full-sort.

$1.68\times$ faster than the best full sort at both sizes. As an anchor, BuRR builds end-to-end at about 70 ns/key at its top recursion level on this machine (roughly 75.5 across all levels). It is a different structure ($r=8$, about 1% space overhead, bumping), whereas our homogeneous configuration spends 7.63 bits/key at 0.81% FPR, about 9.5% overhead. A window sweep from 2^{13} to 2^{20} slots moves totals only between 23.8 and 26.9 ns/key (no tuning cliff), and our registered choice of 2^{16} turns out to have been conservative; 2^{14} does mildly better. (Table 1, Figure 2.)

5 Portability across architectures

We repeated the construction measurements on two ARM cores: an Ampere Altra (Neoverse-N1, NEON) and a Google Axion (Neoverse-V2, SVE), each on eight-or-fewer cores in the cloud. Table 2 gives the 100M-key wall-clock picture. Pleating wins on all three architectures, and by a wider margin on the ARM cores: $2.05\times$ over the reference build on x86, $2.37\times$ on N1, $2.72\times$ on V2. This is what the mechanism predicts (the less aggressively a core hides the dependent-chain misses, the more removing them helps), and the reference banding cost tracks it (46, 114, 86 ns/key on x86/N1/V2). Parallel construction scales near-linearly on the eight-core V2: banding speeds up $1.94/3.78/7.07\times$ at 2/4/8 threads, for a 17.3 ns/key total.

The correctness property carries across architectures without qualification. The solved filter is byte-for-byte identical on all three cores: at every one of the nine (size, seed) points, the Axion fingerprint

Table 2: Construction across architectures, 100M keys, wall-clock ns/key (mean $\pm$ sample stdev over 3 reps). Cloud ARM hypervisors do not expose hardware counters, so cache-miss and IPC figures are reported for x86 only; the ARM columns are timing and output only.

	x86 Raptor Lake	N1 (NEON)	V2 (SVE)
reference	49.6 $\pm$ 1.4	122.8 $\pm$ 0.3	93.7 $\pm$ 0.5
best full sort	40.7 $\pm$ 0.3	64.6 $\pm$ 0.3	44.8 $\pm$ 0.3
partitioned	24.2 $\pm$ 0.2	51.7 $\pm$ 0.3	34.4 $\pm$ 0.0
pleating vs reference	2.05 $\times$	2.37 $\times$	2.72 $\times$

equals both the x86 and the N1 fingerprint, and every strategy on every core agrees with the sequential build. Order-independence is thus also architecture-independence: the same filter, bit for bit, whether banded on AVX2, NEON, or SVE, and whether banded on one thread or eight. Zero false negatives on every run.

6 Order-independence and parallel construction

Proposition 1 (Order-independence) *Let $E = \{(r_k, b_k)\}$ be equations over $\text{GF}(2)^m$ whose rows are linearly independent, each r_k carrying a leading 1 at its start position. Then banding (store each reduced row at the slot of its leading position) followed by back-substitution with all free slots fixed to zero yields the same solution Z under every insertion order of E .*

Proof sketch. The occupied-slot set equals the canonical pivot set of $V = \text{span}\{r_k\}$, which depends only on V ; the stored system is row-equivalent to E ; and pinning the order-independent free coordinates to zero extends E to a full-rank system with a unique solution. (Full argument in the repository, docs/order-independence-proof.md; with dependent rows the enforced subset may differ by order, which the per-run fingerprint gate detects.)

Empirically this held everywhere we looked: across every sequential and parallel run (sizes up to 400 million keys, six different insertion orders, thread counts up to 16), the finished filter came out byte-for-byte identical, checked by hashing the output bits. The practical consequence is larger than it first appears. If the output cannot depend on insertion order, then any reordering scheme, including a parallel one, can prove its own correctness by comparing one checksum against the sequential build. What is normally the hard part of parallel data-structure construction (arguing that no interleaving corrupts the result) becomes a single equality test. Our parallel version assigns disjoint window ranges (split at the same key count) to threads; keys starting in 2^{14} slots of a range boundary switch to a sequential tail (0.015 - 0.226% of keys), following parallel BuRR’s boundary idea. Banding scales 1.81/2.83/4.25/7.56 $\times$ at 2/4/8/16 threads against its own T=1 (Figure 3) (3.5/6.3 $\times$ at 8/16 threads against the best sequential banding; the parallel path carries $\sim 20\%$ defer/copy overhead); what remains is the work we did not parallelize: the counting pass (5.7 ns/key) and back-substitution (3.8 ns/key at 16 threads) are still sequential, and together they now account for most of the 11.9 ns/key total. Both are parallelizable (counting sort trivially; back-substitution as in parallel BuRR); we leave both to future work.

7 Construction prefetch, evaluated

RocksDB ships a software-pipelined construction prefetch behind a comment reading “TODO: verify/validate” and the reference harness enables the same pipeline above 1500 slots. Our no-prefetch control isolates it: 1.5-1.7 $\times$ on unsorted banding (76.9 $\rightarrow$ 46.1 ns/key at 100M). It attacks the start-locating miss but can’t touch the dependent reduction chain, which is why partitioning dominates here while the similar prefetch strategy wins for Bloom construction (§3).

8 Pleating a production engine: RocksDB

The measurements so far use a reference-kernel harness on homogeneous w=64 ribbon. The filter that production actually ships is different: RocksDB’s Standard128RibbonBitsBuilder, a *standard*

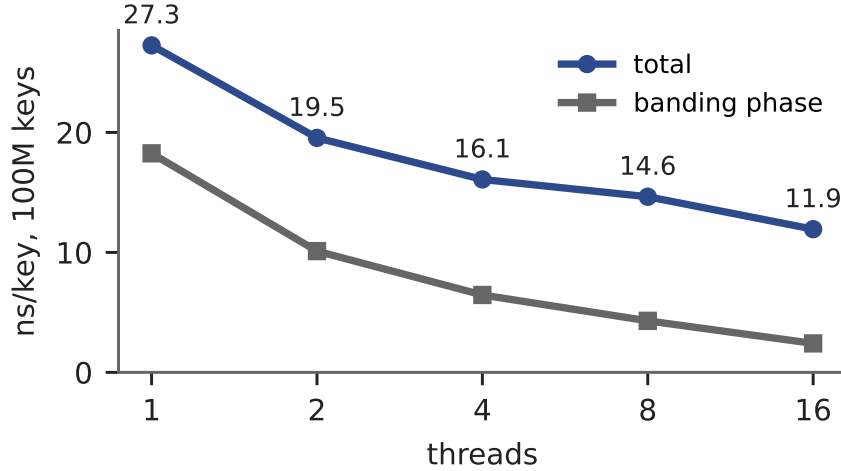


Figure 3: Slot-range parallel construction, 100M keys; every point bit-identical to the sequential build.

(bumped-capable) ribbon at $w=128$ [4]. So we tested whether pleating transfers to it, in RocksDB’s own build path, without changing the filter. The patch is one counting pass over the collected key hashes, inserted immediately before RocksDB’s banding call (`ResetAndFindSeedToSolve`); everything downstream is unmodified engine code. Because banding is order-independent (§6), the finished filter is bit-for-bit identical to the arrival-order build: RocksDB’s own filter tests pass unchanged and the measured false-positive rate is identical to six digits, so correctness is validated by the vendor’s suite, not ours. We gate every measurement on a binary confirmed to contain the patch (object recompiled, binary newer than source, marker string present) after an early build-failure artifact taught us not to trust a stale binary.

Construction microbenchmark (`filter_bench`, the engine’s own tool; 10 bits/key). Table 3 and Figure 4 give the result: at 100M keys per filter pleating builds RocksDB’s ribbon $1.8\times$ faster, and *pleated build cost is flat with filter size* (~ 53 ns/key from 1M to 100M keys) while stock construction degrades from 52 to 94 ns/key as the banding table outgrows cache. Below the cache crossover (a filter of 10^5 keys fits in cache) pleating is a few percent *slower* (the counting pass costs something and there is no locality to recover), exactly as the mechanism predicts.

Why, measured. The reason is the same dependent-miss chain, and we confirm it with hardware counters scoped to RocksDB’s banding call. At 100M keys pleating cuts banding cache-misses from 11.4 to 0.46 per key (a $25\times$ reduction) while the instruction count is unchanged (243 per key, both). Same work, same arithmetic; the only thing that changed is that the working set now fits in cache. Two things differ from the $w=64$ story and both follow from the mechanism. First, standard $w=128$ spends ~ 243 instructions per key against ~ 98 for homogeneous $w=64$ (128-bit GF(2) reductions plus a per-seed rehash), so a larger share of each key’s time is compute that no layout change can remove, which is precisely why the *time* speedup is $1.8\times$ here versus $2.05\times$ for $w=64$, even though the miss reduction is comparable. The memory win is real but diluted by more compute per key. Second, RocksDB’s shipped construction prefetch (§7) attacks the same start-locating miss; pleating subsumes it (pleated construction is unchanged whether the prefetch is on or off), consistent with the homogeneous finding that partitioning makes the prefetch redundant.

End-to-end during compaction (`db_bench`: fill random 30M keys then full compaction, 8-byte values and 256 MB target SSTs so filters are a meaningful share of compaction; three reps). Here the honest accounting matters more than a headline. Filter construction runs during background compaction, so it does not sit on the write path, and write throughput is unchanged. The robust, reproducible result is that the $\sim 2\times$ filter-build speedup survives inside real compaction: banding drops $77.6 \pm 1.5 \rightarrow 44.2 \pm 6.1$ ns/key (means of 3; the isolated, uninstrumented build shows the fuller $2.0\times$, compressed here by the statistics instrumentation). Filter banding is $\sim 23\%$ of compaction CPU in this configuration (1.83 s of 7.9 s), so halving it reclaims a real slice. We are deliberately careful

Table 3: Pleating RocksDB’s shipped Standard128 ribbon (filter_bench, 10 bits/key, i9-14900HX; means of 3). False-positive rate identical stock vs pleated at every size (bit-identical filter). Banding cache-misses/key from hardware counters scoped to the banding call.

keys/filter	stock ns/key	pleated ns/key	speedup
10^5 (cache-resident)	57.6	59.4	$0.97\times$
10^6	55.9	54.4	$1.03\times$
10^7	88.7	54.7	$1.62\times$
10^8 (paper regime)	97.4	54.8	$1.78\times$
banding misses/key @ 10^8	11.5	0.47	—

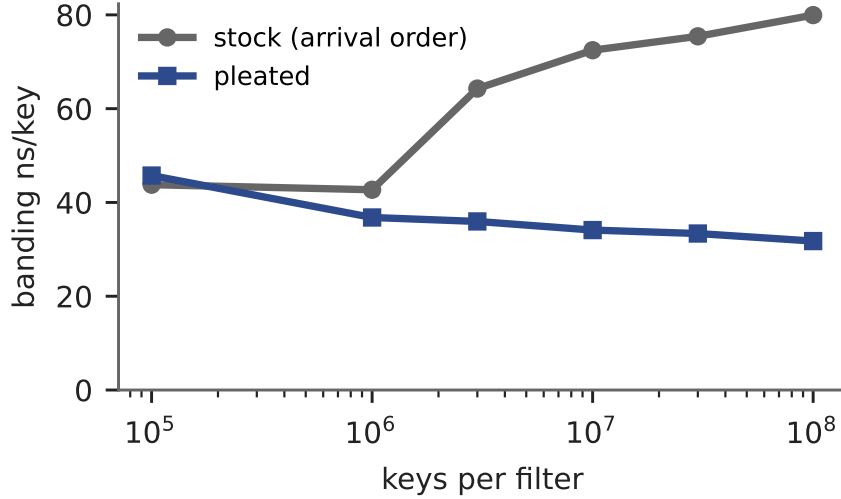


Figure 4: RocksDB Standard128 ribbon construction vs. filter size. Pleated build cost stays flat (banding cache-resident per window); stock degrades as the table outgrows cache. Bit-identical filter at every point.

about the whole-compaction number: total compaction CPU falls $\sim 9\%$ ($7.9 \pm 0.8 \rightarrow 7.2 \pm 0.4$ s), but that difference is of the same order as the run-to-run variance of compaction itself (which is nondeterministic), so we report it as directional, not tightly bounded. The effect also grows with per-SST filter size and vanishes below the cache crossover, so its magnitude is a function of SST sizing; this is the filter-favorable end of that range, labelled as such. The transferable claim is narrow and solid: *pleating removes a real, scale-growing slice of compaction CPU from a production LSM engine at zero cost to the output or read path*, and the mechanism that does it is the one this paper isolates.

9 Related work

Sorted banded elimination: Dietzfelbinger and Walzer [7], BuRR [3], parallel BuRR [5]. Arbitrary-order banding and production engineering: ribbon [2], RocksDB [4]. Cache-conscious filter design: buffered Bloom filters [10], radix-partitioned filters [8], binary-fuse segment sort [9], xor-filter buffered scatter [11]; the generic two-phase partition-then-accumulate pattern appears as propagation blocking [12] with write-combining from Wassenberg and Sanders [13]. SIMD probe-side layout: Polychroniou and Ross [14], Lang et al. [15], split-block Bloom [16]. Our delta to BuRR is precise: BuRR sorts fully (ips2ra) to enable threshold-based bumping and tighter space; we show that for homogeneous ribbon a counting-pass partition captures 98% of the sort’s locality at $4\times$ less reorder cost, with output neutrality making the transformation and its parallelization verifiable.

10 Limitations and threats to validity

We characterize the miss mechanism with hardware counters on x86 only; the two ARM replications (§5) confirm the wall-clock speedup and the bit-identical output, but cloud ARM hypervisors do not expose the PMU, so the dependent-miss mechanism itself is not directly measured on ARM. It is, however, exactly what the ARM timing behavior implies. A bare-metal ARM run would close this, and is the one measurement we would add. We accelerate the *homogeneous* ribbon ($\sim 9.5\%$ space overhead at $r=7$) in comparison to BuRR (which is sub-1% overhead); the comparison with BuRR is an anchor, not an apples-to-apples comparison. Sort baselines use BuRR’s own sequential `ips2ra`, but not its parallel form. Our parallel comparison is limited to banding, and its headline ratios are against the parallel path’s own single-thread run. The order-independence proposition is proved for linearly independent rows, and the dependent case is handled empirically by the fingerprint gate. The RocksDB integration (§8) demonstrates the construction speedup on the production $w=128$ kernel and during real compaction; its end-to-end magnitude depends on SST sizing and we report a filter-favorable configuration, so the whole-workload effect on a values-heavy, default-SST deployment would be smaller.

11 Conclusion

On the modern OoO x86, memory-layout optimization for filters pays in exactly one place that we could find — and it is the place that matters: the dependent-miss chains of ribbon construction, the very cost that keeps space-optimal filters out of production write paths. A one-pass partition closes 98% of the gap the literature closes with a full sort, at a quarter of the price; the order-independence makes it safe and its parallelization checkable; and 16 threads build ribbon filters as fast as one thread inserts into a Bloom filter. The same one-pass partition, dropped into RocksDB’s shipped $w=128$ ribbon builder, makes its construction $1.8\times$ faster with a bit-identical filter (§8): the mechanism transfers to production unchanged. The two nulls are not failures of the method but the map that located the win.

Artifacts. All protocols, amendments, raw measurements, and integrity tooling are in the accompanying repository; `reproduce.sh` re-derives every number in this paper from committed raw artifacts. A production implementation of pleated construction (differentially gated byte-for-byte against the reference kernel) is released as the `pleat` crate at <https://github.com/integrallis/pleat>.

References

- [1] Burton H. Bloom. Space/time trade-offs in hash coding with allowable errors. *CACM*, 13(7), 1970.
- [2] Peter C. Dillinger and Stefan Walzer. Ribbon filter: Practically smaller than bloom and xor. *arXiv:2103.02515*, 2021.
- [3] Peter C. Dillinger, Lorenz Hübschle-Schneider, Peter Sanders, and Stefan Walzer. Fast succinct retrieval and approximate membership using ribbon. In *SEA*, 2022.
- [4] Peter C. Dillinger. Ribbon filter. RocksDB blog, rocksdb.org/blog/2021/12/29/ribbon-filter.html, 2021.
- [5] Christopher Becht, Hans-Peter Lehmann, and Peter Sanders. Brief announcement: Parallel construction of bumped ribbon retrieval. *arXiv:2411.12365*, 2024.
- [6] Antoine Limasset. ZOR filters. *arXiv:2602.03525*, 2026.
- [7] Martin Dietzfelbinger and Stefan Walzer. Efficient Gauss elimination for near-quadratic matrices with one short random block per row, with applications. In *ESA*, 2019.
- [8] Tobias Schmidt, Maximilian Bandle, and Jana Giceva. A four-dimensional analysis of partitioned approximate filters. In *PVLDB 14(11)*, 2021.
- [9] Thomas Mueller Graf and Daniel Lemire. Binary fuse filters: Fast and smaller than xor filters. *ACM JEA*, 27, 2022.

- [10] Mustafa Canim, George A. Mihaila, Bishwaranjan Bhattacharjee, Christian A. Lang, and Kenneth A. Ross. Buffered Bloom filters on solid state storage. In *ADMS@VLDB*, 2010.
- [11] Thomas Mueller Graf and Daniel Lemire. Xor filters: Faster and smaller than bloom and cuckoo filters. *ACM JEA*, 25, 2020.
- [12] Scott Beamer, Krste Asanović, and David Patterson. Reducing PageRank communication via propagation blocking. In *IPDPS*, 2017.
- [13] Jan Wassenberg and Peter Sanders. Engineering a multi-core radix sort. In *Euro-Par*, 2011.
- [14] Orestis Polychroniou and Kenneth A. Ross. Vectorized bloom filters for advanced simd processors. In *DaMoN*, 2014.
- [15] Harald Lang, Thomas Neumann, Alfons Kemper, and Peter Boncz. Performance-optimal filtering: Bloom overtakes cuckoo at high throughput. *PVLDB*, 12(5), 2019.
- [16] Jim Apple. Split block bloom filters. arXiv:2101.01719; Apache Parquet format specification, 2021.